

UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA
Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Laboratorio No. 5 · Integridad referencial y API Keys

ESPACIO ACADÉMICO:
Seguridad en el Desarrollo de Software

PRESENTADO A:
Sergio Arley Puerto Moreno

AUTORES:
Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. Qué hicimos

El laboratorio pedía dos cosas para nuestro proyecto, la API del Teatro Maldonado de Tunja. Primero, impedir que se borre un recurso del que dependen otros. Segundo, pedir una clave para poder usar la API.

El Bloque 2, las pruebas SCA, SAST y DAST, lo entregamos el 16 de septiembre y va adjunto aparte. Su checkpoint pedía tener el DELETE protegido y en ese momento solo lo teníamos en funciones; con el Bloque 1 quedaron los cuatro.

Repositorio: https://github.com/JuanK2550/API-TEATRO

2. Bloque 1. Integridad referencial

Como los datos están en memoria, nada impedía borrar una función que ya tenía boletas vendidas. Esas boletas quedaban apuntando a un id que ya no existía, o sea un registro huérfano. Hicimos lo mismo que después hará PostgreSQL con `ON DELETE RESTRICT`: antes de borrar, preguntar si alguien depende de ese registro, y si depende responder 409.

Revisamos los cinco recursos y protegimos cuatro borrados: asistentes con boletas, eventos con funciones, localidades con boletas o con tarifas, y funciones con boletas. La localidad es el único con dos comprobaciones, porque puede estar relacionada de dos maneras.

Cancelar no es lo mismo que eliminar

Esta parte fue la que más nos hizo pensar. Ya teníamos una función que no cuenta las boletas canceladas, y para el negocio está bien: si cancelas tu boleta, la butaca queda libre y otro la compra.

Pero para la integridad referencial la pregunta es otra. Una boleta cancelada sigue siendo un registro que existe, y si borramos la función queda apuntando a la nada. Por eso las funciones nuevas cuentan todas las boletas. Terminamos con dos preguntas parecidas pero distintas: una mira si la butaca está libre ahora, la otra si alguna vez se vendió.

El cambio tuvo efecto: una función cuya única boleta está cancelada ya no se deja borrar. Se nos rompieron cuatro pruebas y las actualizamos.

Cómo quedó

Agregamos cinco funciones, tres en el servicio de boletas y dos en el de funciones. Los cuatro borrados preguntan siempre en el mismo orden: si el recurso no existe, 404; si tiene relaciones, 409; si pasa las dos, se borra y devuelve 200. El id mal escrito lo rechaza antes el validador con 400.

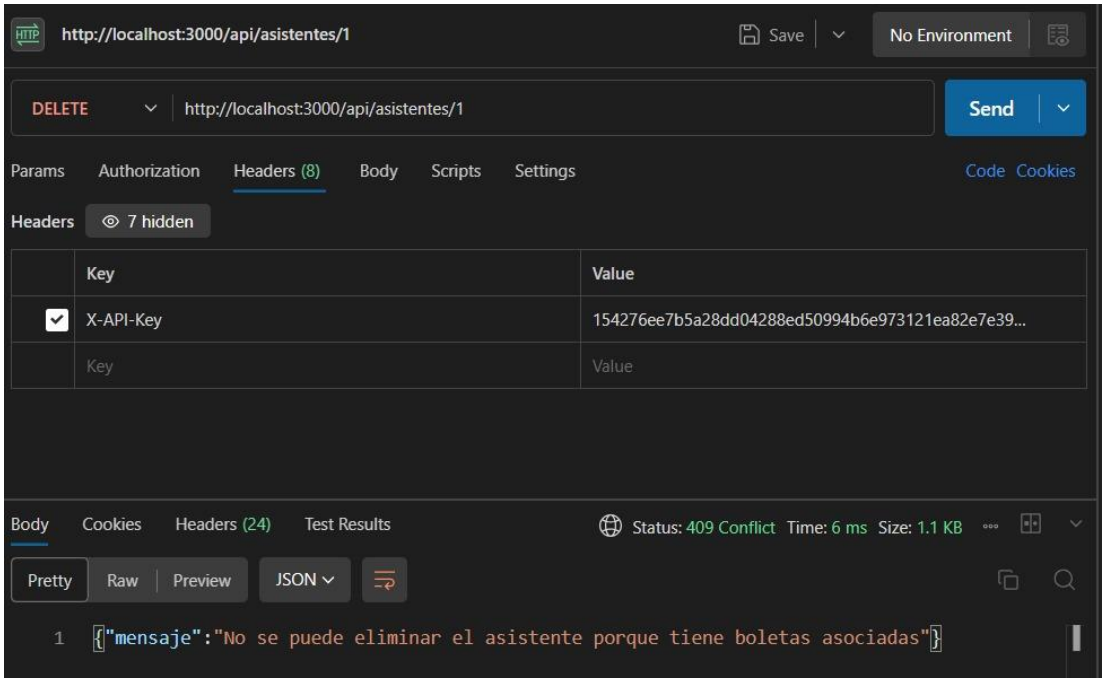


Figura 1. Asistente con boletas. Responde 409.

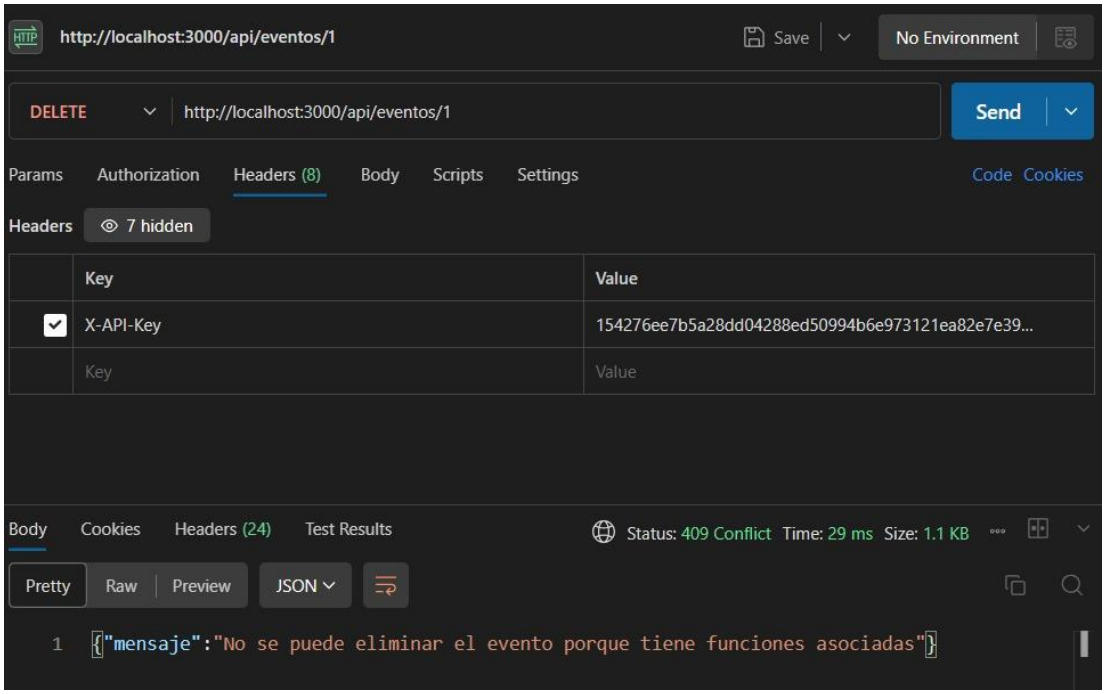


Figura 2. Evento con funciones. Responde 409.

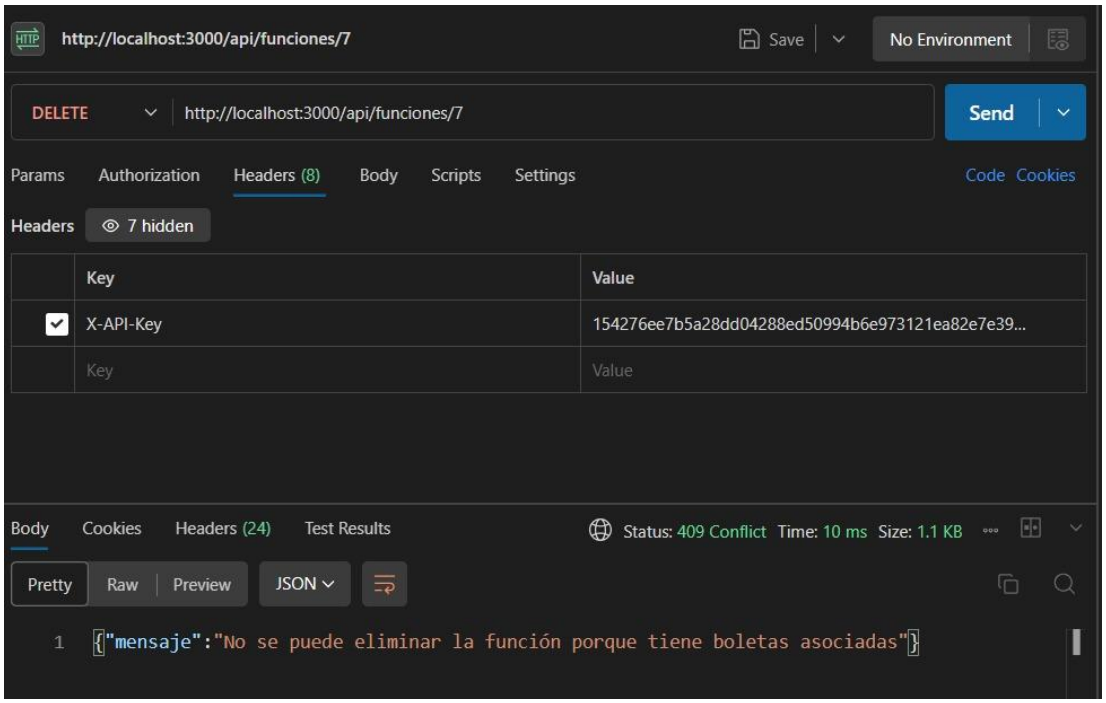


Figura 3. Función 7, cuya única boleta está cancelada. Responde 409.

La figura 3 es la que muestra el criterio: esa función no tiene ninguna boleta activa y aun así no se deja borrar.

Para el caso contrario tuvimos que crear datos, porque todos los recursos que vienen cargados tienen relaciones.

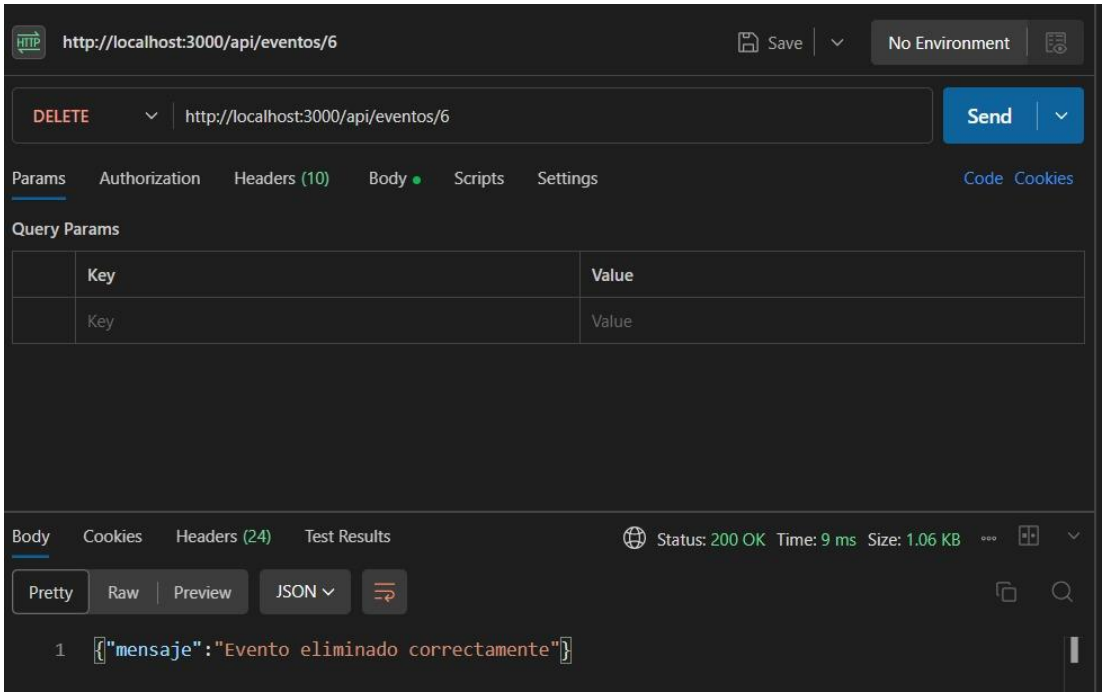


Figura 4. Un evento recién creado, sin relaciones. Se borra y responde 200.

Checkpoint

Las siete comprobaciones del laboratorio las hicimos contra el servidor, con 26 casos y ningún fallo: borrar con relaciones da 409, borrar sin relaciones da 200, un id que no existe da 404, uno mal escrito da 400, el 404 sale siempre antes que el 409, una relación cancelada sigue bloqueando, y no quedaron referencias rotas.

Del quinto punto hay que aclarar algo: no se puede probar con un id que a la vez no exista y tenga relaciones. Lo que comprobamos es que un recurso ya borrado da 404 y nunca llega a mirar las relaciones.

3. Bloque 3. Protección con API Key

Hasta aquí cualquiera que llegara a `/api` podía hacer lo que quisiera.
Ahora hay que mandar la cabecera `X-API-Key`. Vale aclarar hasta dónde llega: identifica al cliente que usa la API, no a una persona. Todavía no distingue entre alguien que compra boletas y un administrador; eso viene después con usuarios y JWT.

La clave

No inventamos una clave tipo 123456. Node trae un generador seguro y sacamos 32 bytes en hexadecimal.



Figura 5. La clave generada con crypto.randomBytes.

La clave real va en. env, que está en `.gitignore` y no se sube. El `.env.example` sí se sube, pero con un texto de relleno. Así el que clone el proyecto sabe qué variables necesita sin quedarse con el secreto.



Figura 6. El .env con la clave real.

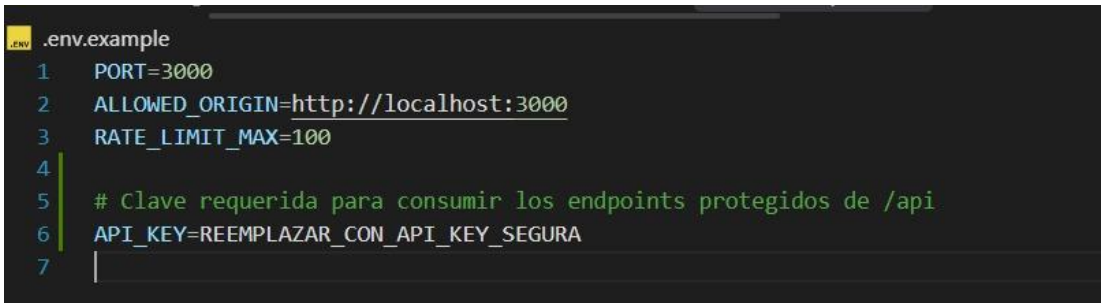


Figura 7. El .env.example. Las mismas variables, pero sin la clave.

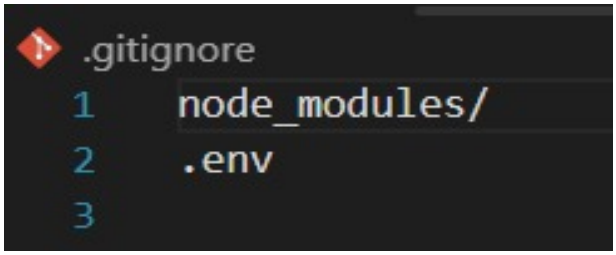


Figura 8. El .gitignore, con .env adentro.

La clave de estas capturas ya no la usamos, generamos otra después de tomar las fotos.

El middleware

Creamos `apiKey.middleware.js`. Si falta la cabecera responde 401 con "API Key requerida"; si llega pero está mal, 401 con "API Key inválida"; si está bien, deja pasar. Va sobre `/api`, después del límite de peticiones y antes de las rutas, así que una petición sin clave se corta antes del validador y nunca llega al código que crea datos.

Para comparar la clave usamos `crypto.timingSafeEqual` en vez de `===`. Una comparación normal se detiene apenas encuentra un carácter distinto, así que demora un poquito más cuando el atacante acierta los primeros caracteres, y midiendo esos tiempos se puede ir adivinando la clave letra por letra. `timingSafeEqual` compara todo siempre y demora lo mismo.

Pruebas

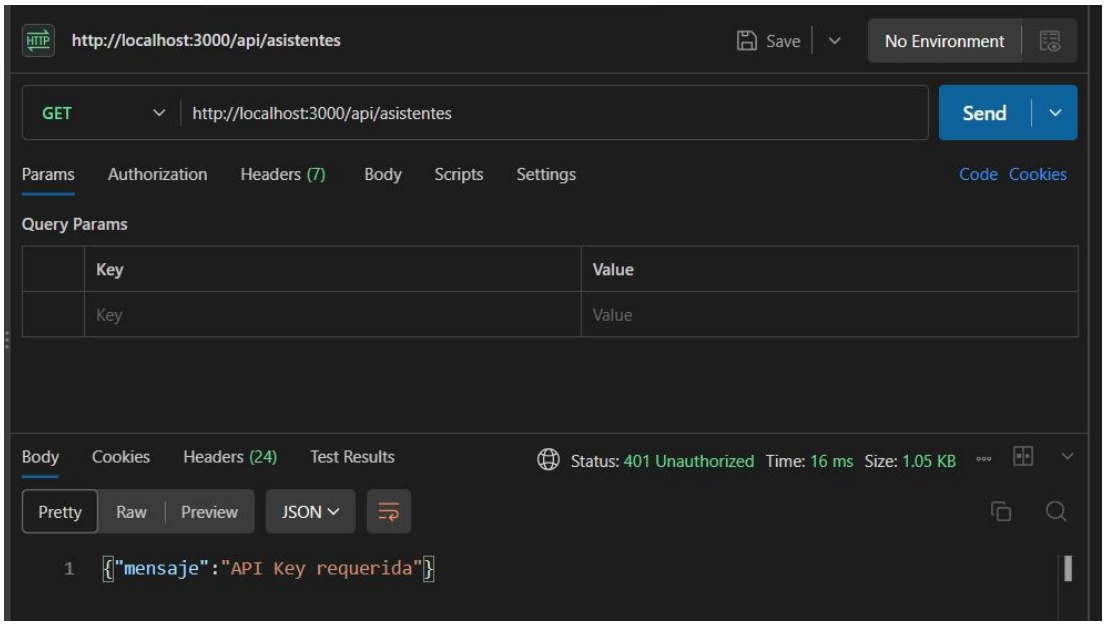


Figura 9. Sin la cabecera. Responde 401, "API Key requerida".

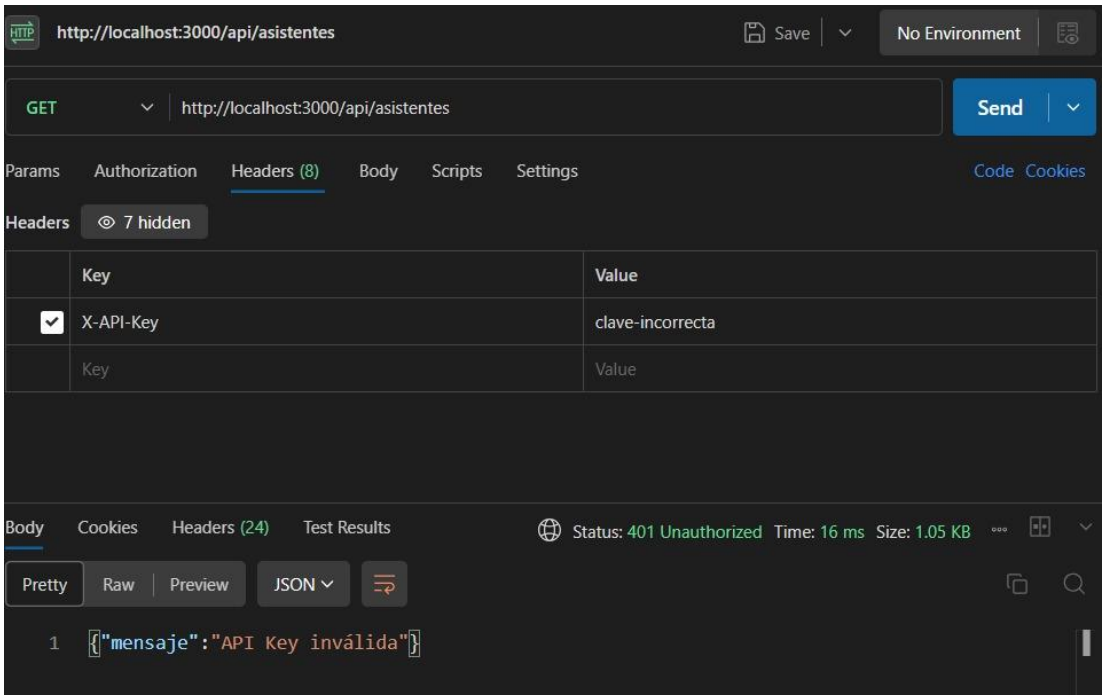


Figura 10. Con una clave inventada. Responde 401, "API Key inválida". El mensaje cambia según si falta la clave o si está mal.

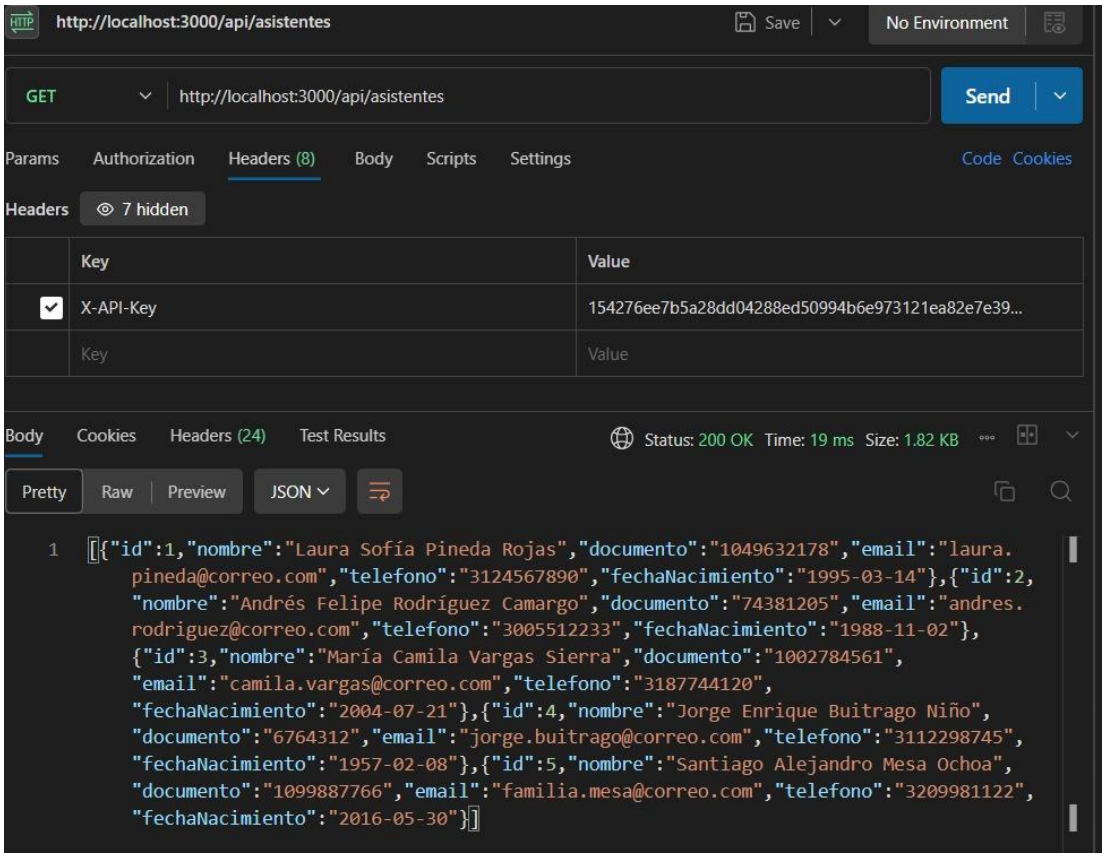


Figura 11. Con la clave buena en la cabecera. Responde 200.

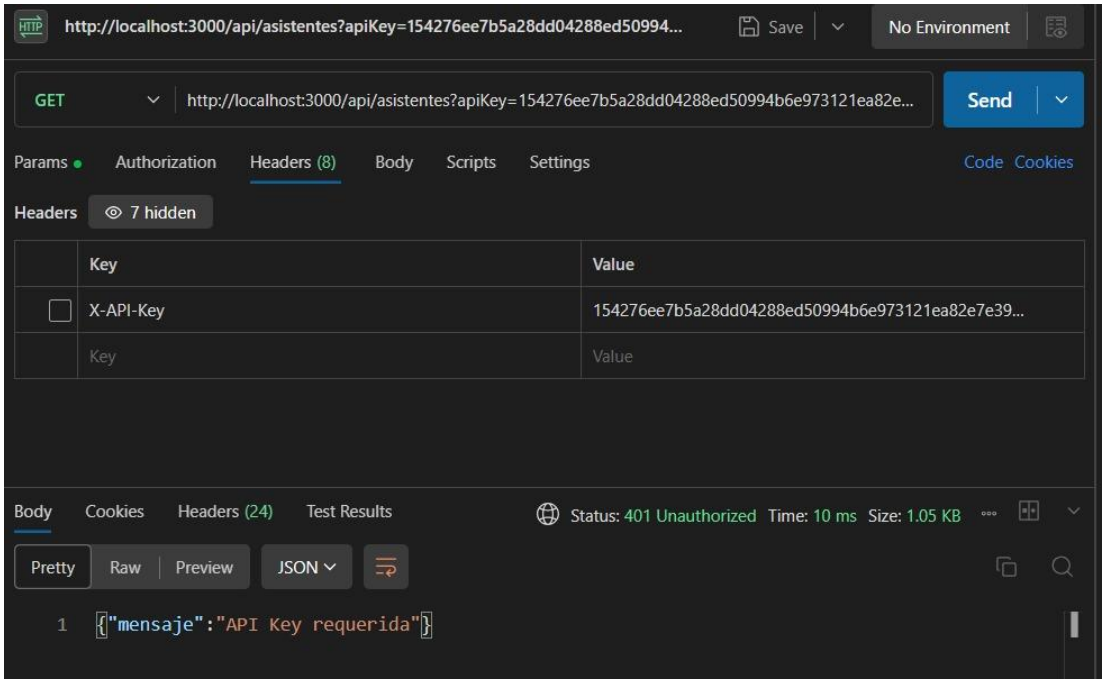


Figura 12. La clave buena, pero mandada en la URL. Responde 401.

La figura 10 es a propósito. La clave es la correcta, pero va en la barra de direcciones y el middleware solo lee la cabecera. Las URL quedan guardadas en el historial, en los logs del servidor y en los proxies, así que un secreto puesto ahí termina copiado donde uno no controla.

Swagger

Al montar el middleware, Swagger dejó de servir para probar: todo daba 401 porque no sabía que tenía que mandar la cabecera. Lo declaramos en la configuración de OpenAPI y apareció el botón Authorize con un candado en cada endpoint.

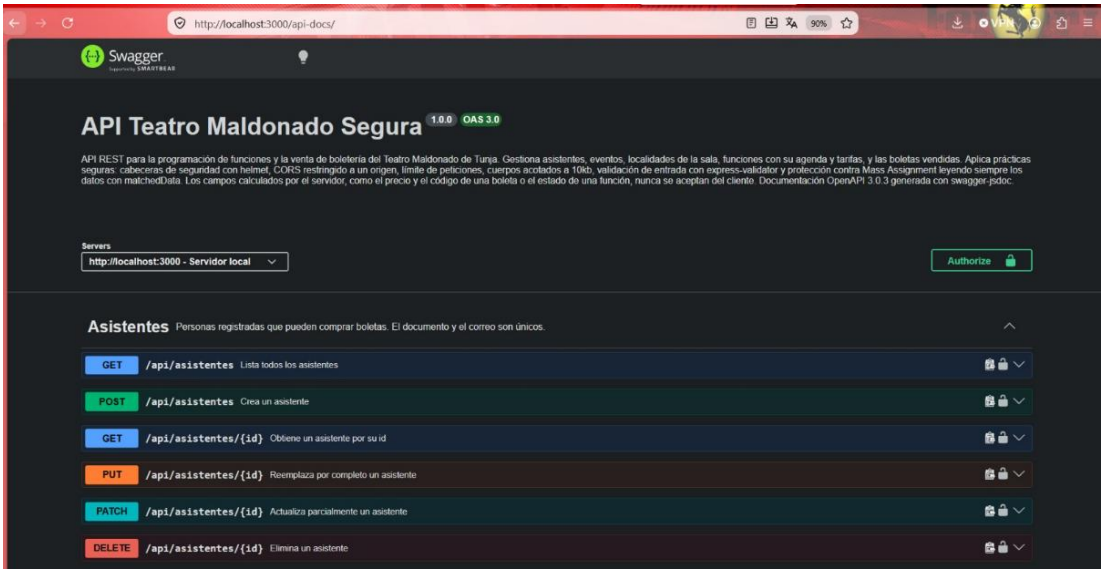


Figura 13. Swagger con el botón Authorize y los candados.

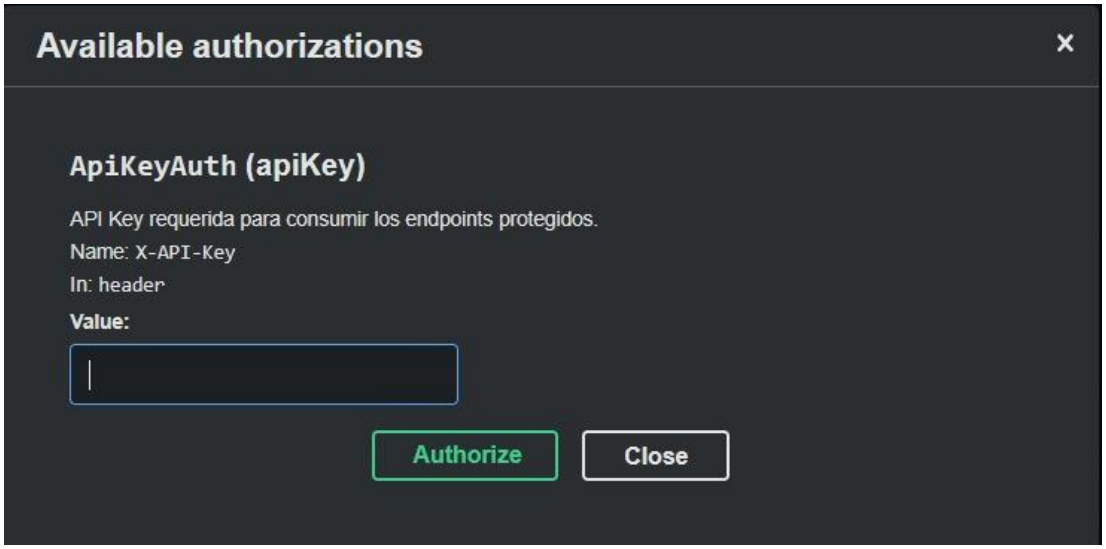


Figura 14. El diálogo donde se pega la clave. Muestra el nombre de la cabecera y dónde viaja.

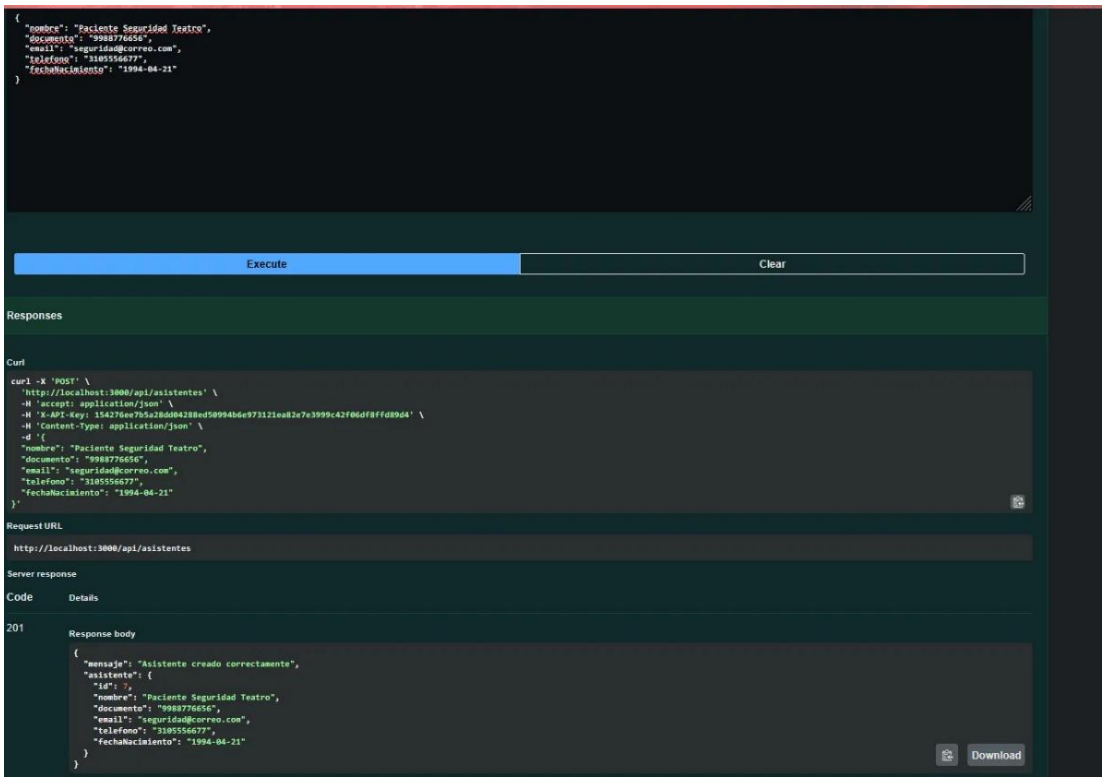


Figura 15. Creamos un asistente ya autorizados. Responde 201 y en el curl se ve la cabecera.

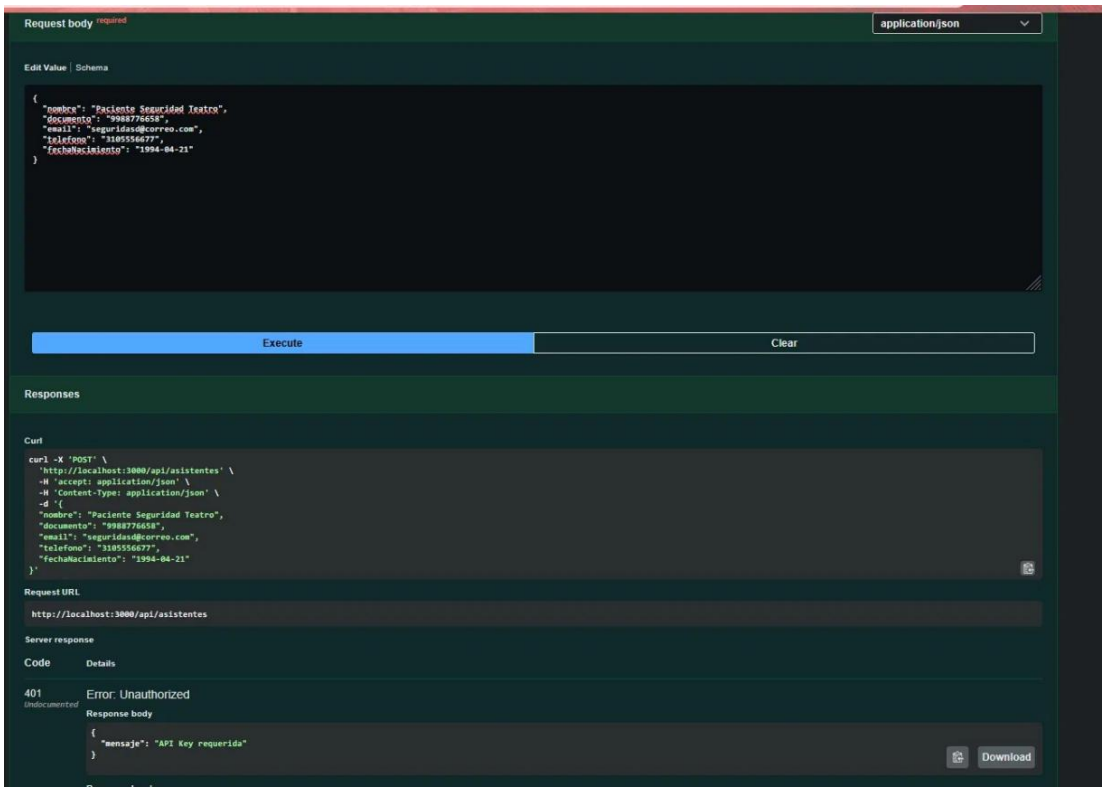


Figura 16. La misma petición después del Logout. Responde 401 y la cabecera ya no aparece.

Esas dos muestran que Swagger no se salta la seguridad, se comporta como cualquier otro cliente. Y que la petición sin clave no llega al controlador: después del 401 miramos la lista y no había ningún asistente con ese documento.

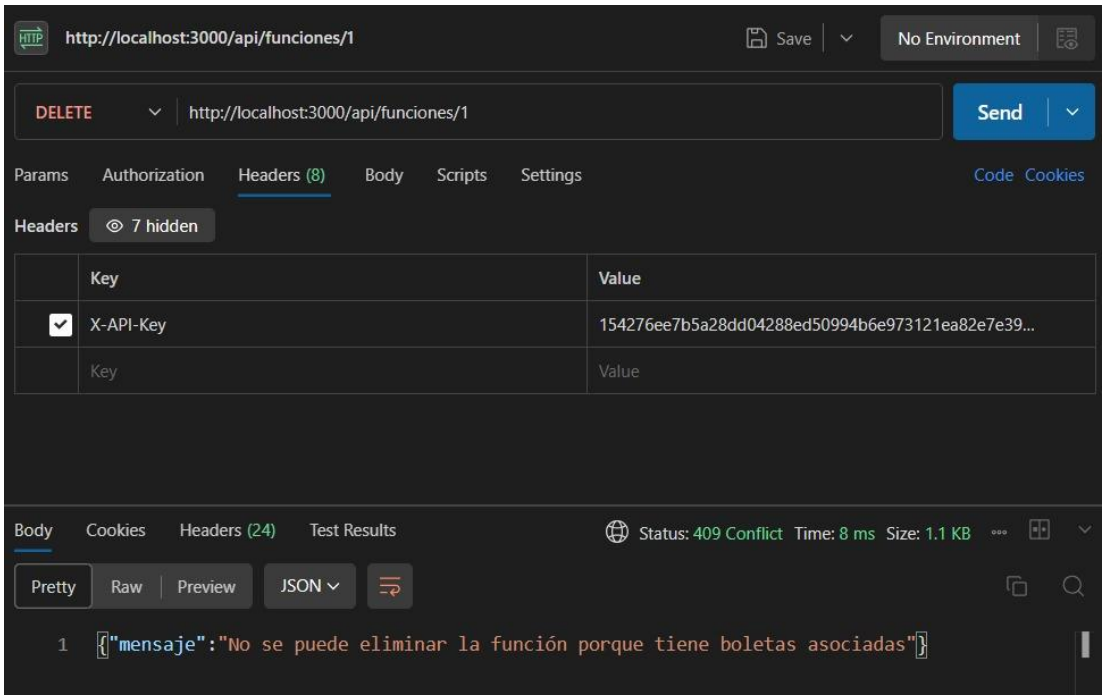


Figura 17. Borrar una función con boletas, con la clave correcta. Responde 409.

Estar autenticado no da permiso para romper el sistema. La clave dice quién usa la API, pero las reglas de negocio siguen aplicándose después.

Checkpoint

Sin clave 401, clave mala 401, clave buena 200, Swagger con Authorize 200 y 401 tras el Logout, clave en la URL 401, y borrado protegido 409.

Agregamos un caso que no estaba pedido: una clave mala pero del mismo largo que la real. Es el único que de verdad prueba la comparación byte por byte, porque si los largos son distintos la función corta antes de llegar ahí.

4. Después del cambio

Volvimos a pasar Semgrep y npm audit. Semgrep corrió 73 reglas sobre 31 archivos sin hallazgos, y npm audit no encontró vulnerabilidades. El bloque no agregó dependencias porque `crypto` ya viene con Node.

Un detalle que vale anotar: Semgrep solo revisa archivos que git ya tenga registrados, y el middleware nuevo todavía no estaba en ningún commit. Lo pasamos aparte
y también salió limpio. Sin esa segunda pasada habríamos creído que estaba revisado cuando no lo estaba.

Al montar el middleware se nos rompieron 218 de las 224 pruebas que teníamos, todas las que le pegan a `/api`. Nos pareció buena señal: si solo se hubieran roto algunas, querría decir que quedaron endpoints sin proteger. Las arreglamos agregando la cabecera a los siete scripts, que leen la clave del `.env`.

5. Repositorio

Todo el proyecto se encuentra disponible en el repositorio de GitHub: https://github.com/JuanK2550/API-TEATRO. El repositorio contiene cuatro commits, correspondientes a los tres bloques de desarrollo y a un commit adicional en el que se incorporaron los scripts de pruebas que anteriormente se encontraban fuera del proyecto.